

Programação Avançada

Aula 09: Algoritmo de Dijkstra

Prof. Eduardo Corrêa Gonçalves

20/05/2026

Sumário

Introdução

Entendendo a Saída do Algoritmo

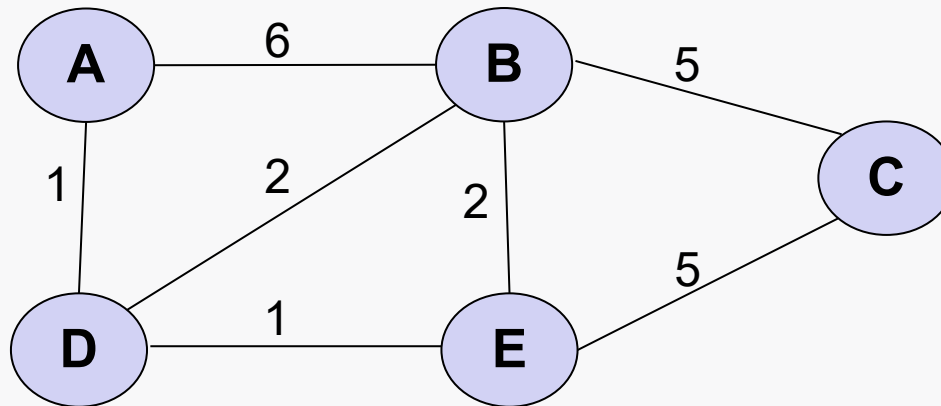
Exemplo Passo a Passo

Pseudocódigo

Introdução (1/2)

- Algoritmo de Dijkstra

- É um algoritmo para **explorar rotas** entre pares de nós em um grafo valorado.
- Mais especificamente:
 - Usado para encontrar **caminho mais curto** (*shortest-path*) entre um determinado nó e todos os outros do grafo.

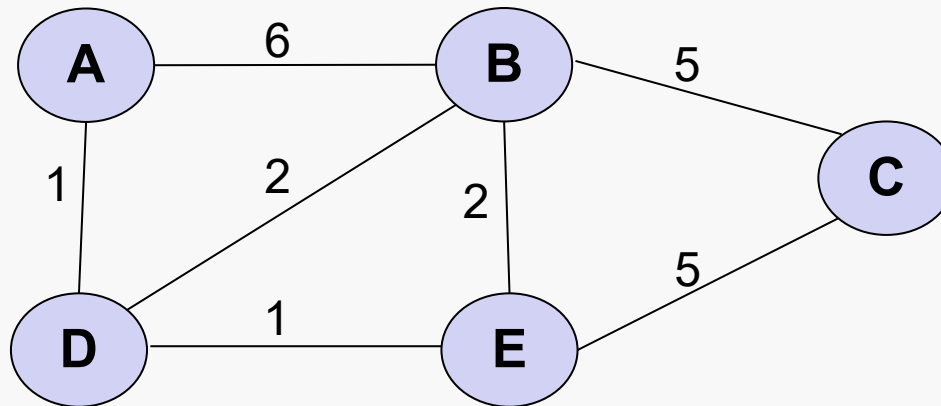


- Exemplo da aula retirado de: <https://www.youtube.com/watch?v=pVfj6mxhdMw>

Introdução (2/2)

- Algoritmo de Dijkstra

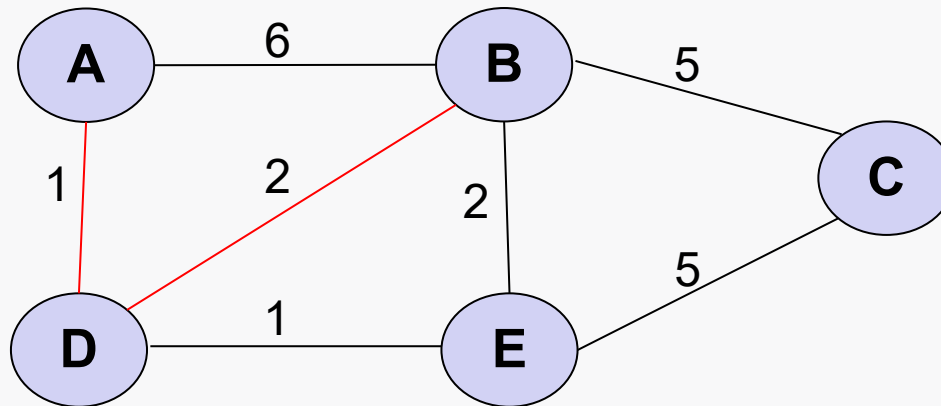
- Por exemplo, para chegar de A até B, existem 4 caminhos possíveis:
 - A-B: distância 6
 - A-D-B: distância 3
 - A-D-E-B: distância 4
 - A-D-E-C-B: distância 12



Introdução (2/2)

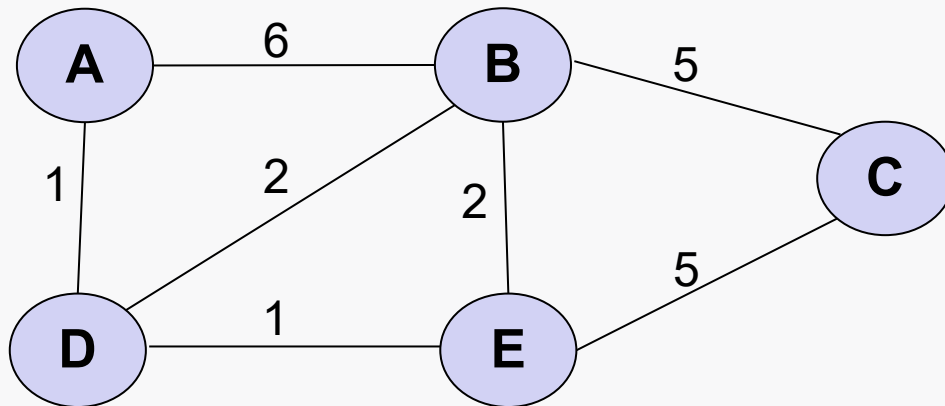
- Algoritmo de Dijkstra

- Por exemplo, para chegar de **A** até **B**, existem 4 caminhos possíveis:
 - **A-B**: distância 6
 - **A-D-B**: distância 3
 - **A-D-E-B**: distância 4
 - **A-D-E-C-B**: distância 12
- O caminho mais curto é **A-D-B**.



Exemplo – Saída do Algoritmo (1/7)

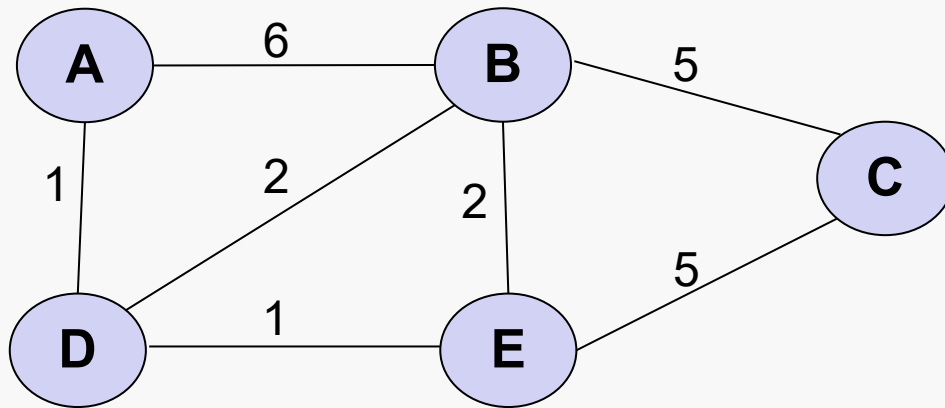
- Caminho mais curto de A a todos os outros vértices
 - Ao final da execução, o algoritmo gerará a tabela abaixo como saída.
 - A tabela é simples, mas **contém tudo que precisamos saber!**



Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

Exemplo – Saída do Algoritmo (2/7)

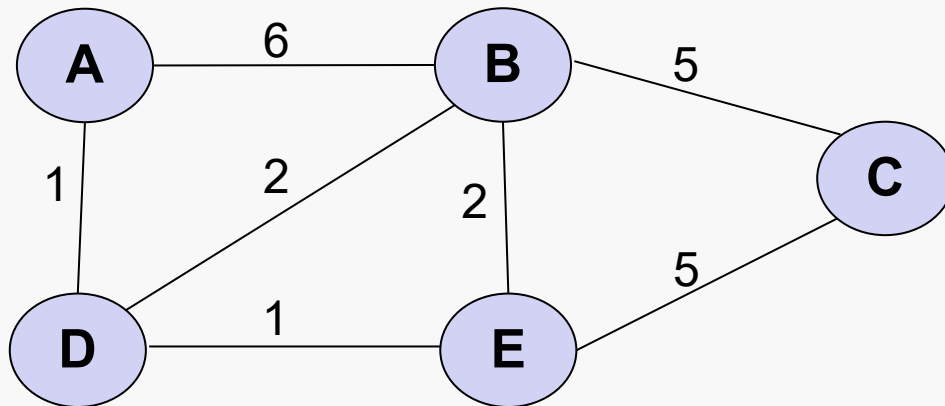
- Caminho mais curto de A aos demais vértices
 - Ao final da execução, o algoritmo gerará a tabela abaixo como saída.
 - Aqui temos o valor da distância mais curta de **A** aos demais vértices



Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

Exemplo – Saída do Algoritmo (3/7)

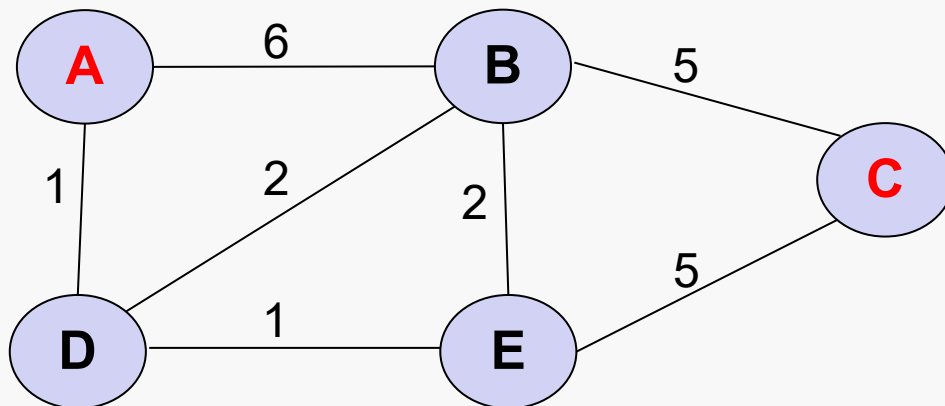
- Caminho mais curto de A aos demais vértices
 - Ao final da execução, o algoritmo gerará a tabela abaixo como saída.
 - E aqui a sequência mais curta de vértices de **A** aos demais.
 - É o caminho mais curto!



Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

Exemplo – Saída do Algoritmo (4/7)

- Caminho mais curto de A aos demais vértices
 - **Exemplo** – caminho mais curto de **A** até **C**

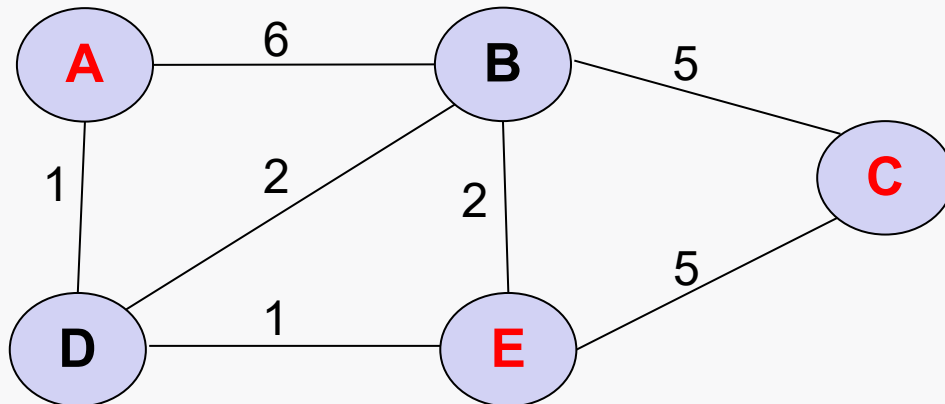


Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

Exemplo – Saída do Algoritmo (5/7)

- Caminho mais curto de A aos demais vértices
 - **Exemplo** – caminho mais curto de **A** até **C**
 - chegamos a **C** via **E**

- Caminho: **E-C**



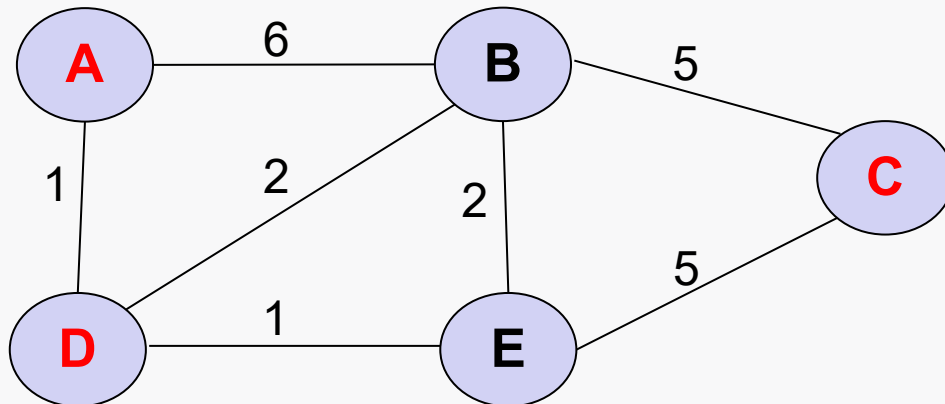
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

Exemplo – Saída do Algoritmo (6/7)

- Caminho mais curto de A aos demais vértices

- **Exemplo** – caminho mais curto de **A** até **C**
 - chegamos a **C** via **E**
 - chegamos a **E** via **D**

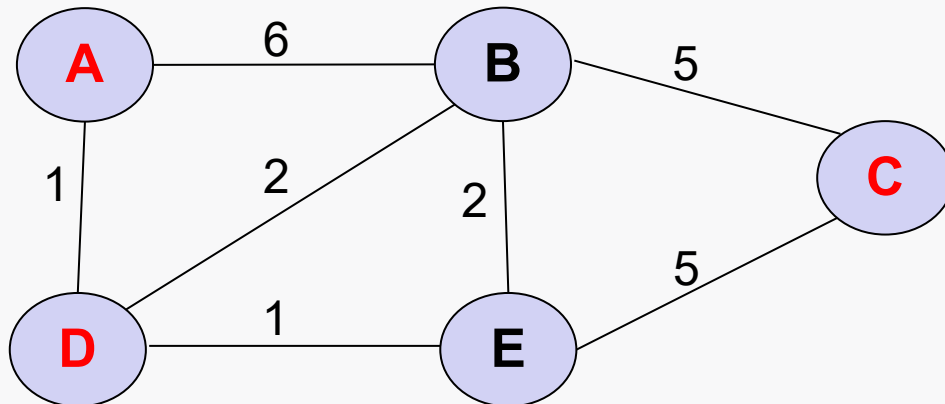
- Caminho: **D-E-C**



Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

Exemplo – Saída do Algoritmo (7/7)

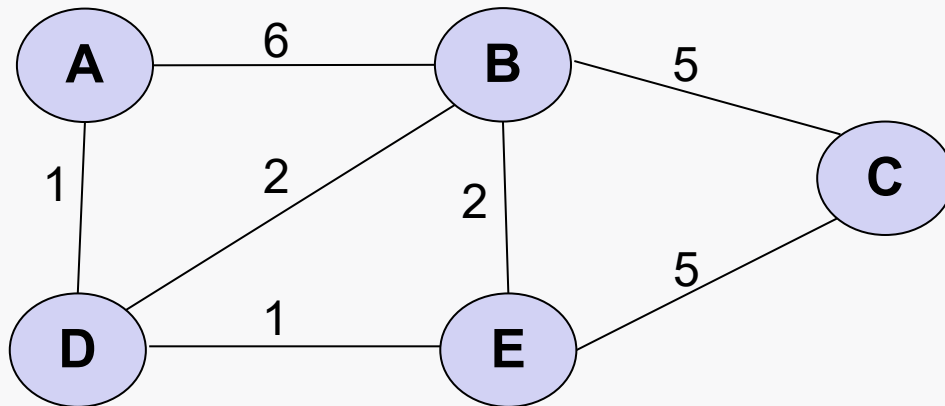
- Caminho mais curto de A aos demais vértices
 - **Exemplo** – caminho mais curto de A até C
 - chegamos a C via E
 - chegamos a E via D
 - chegamos a D diretamente de A
 - *Pronto! Esse é o caminho completo!*
 - Caminho: **A-D-E-C**



Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

Dijkstra em Ação (0/7)

- Demonstração da execução completa do algoritmo
- **PASSO 0**: Para começar, 3 estruturas de dados são criadas
 - **tabela de resultados**: inicialmente *vazia*
 - **visitados**: lista de nós visitados, inicialmente *vazia*.
 - **não-visitados**: lista de nós não-visitados, inicialmente contendo todos



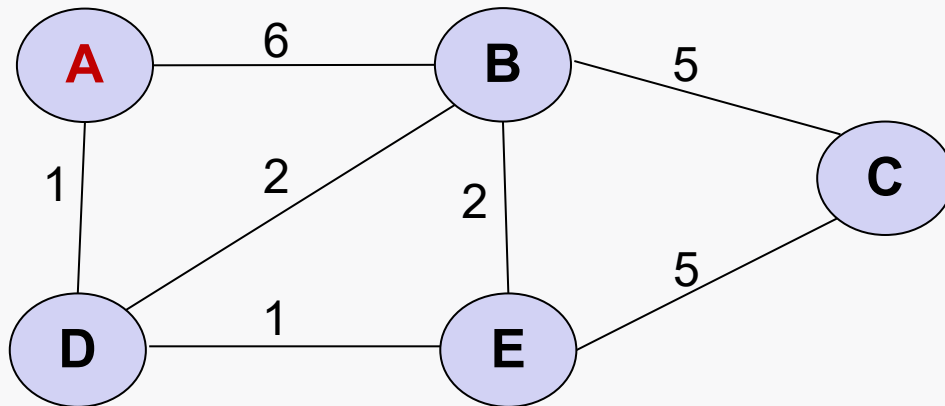
Vértice	Menor Dist. de A	Vértice Anterior
A		
B		
C		
D		
E		

visitados: []

não-visitados: [A, B, C, D, E]

Dijkstra em Ação (1/7)

- PASSO 1**: Tratamento do vértice inicial, neste caso **A**.
 - Dist. A para A = 0 (*óbvio, mas necessário formalizar o algoritmo*)
 - Dist. A para outros vértices = desconhecida, logo ∞



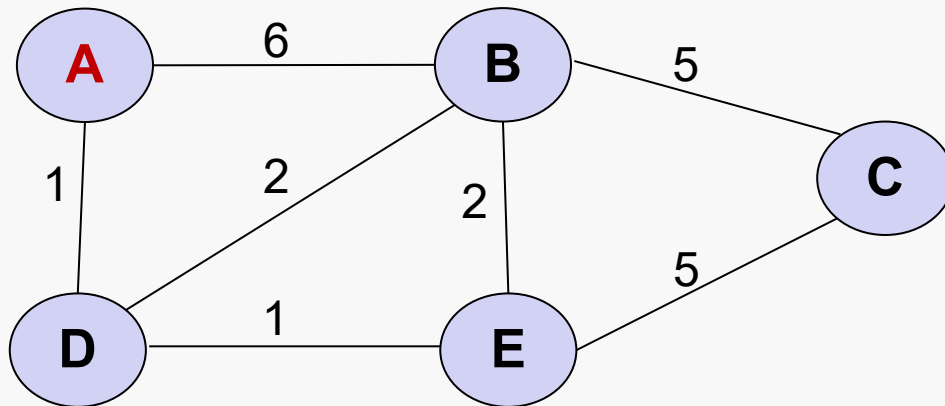
Vértice	Menor Dist. de A	Vértice Anterior
A		
B		
C		
D		
E		

visitados: []

não-visitados: [A, B, C, D, E]

Dijkstra em Ação (1/7)

- **PASSO 1:** Tratamento do vértice inicial, neste caso **A**.
 - Dist. A para A = 0
 - Dist. A para outros vértices = desconhecida, logo ∞
- Assim, a tabela é atualizada.



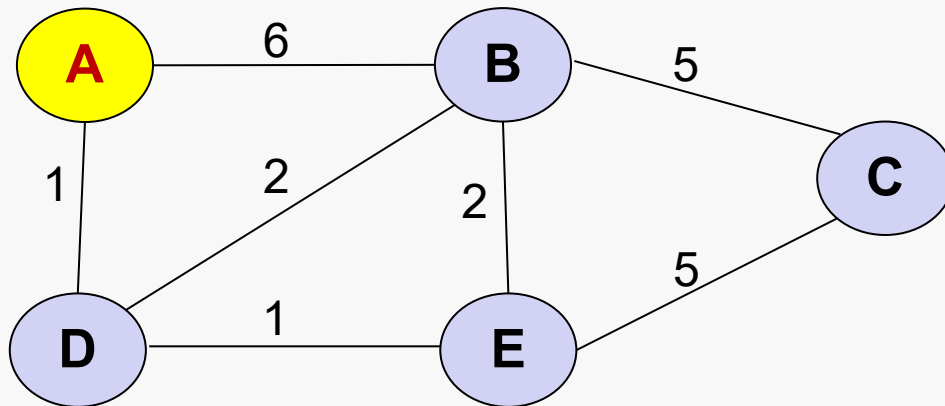
Vértice	Menor Dist. de A	Vértice Anterior
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

visitados: []

não-visitados: [A, B, C, D, E]

Dijkstra em Ação (2/7)

- **PASSO 2:** visitar o vértice não visitado que possuir a menor distância conhecida do vértice inicial.
- No momento, trata-se do próprio vértice inicial **A**.
 - “A” se torna o “nó atual”.



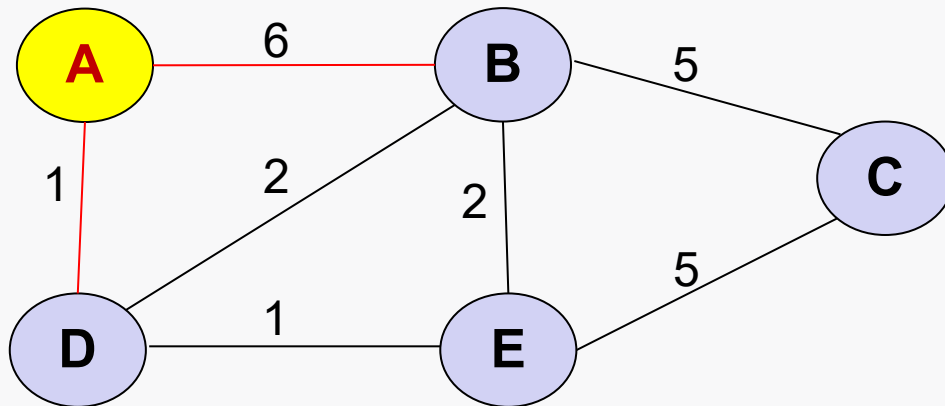
Vértice	Menor Dist. de A	Vértice Anterior
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

visitados: []

não-visitados: [A, B, C, D, E]

Dijkstra em Ação (2/7)

- **PASSO 3:** Examina-se os vizinhos não-visitados do nó atual.
 - O nó atual é A e seus vizinhos não-visitados são B e D



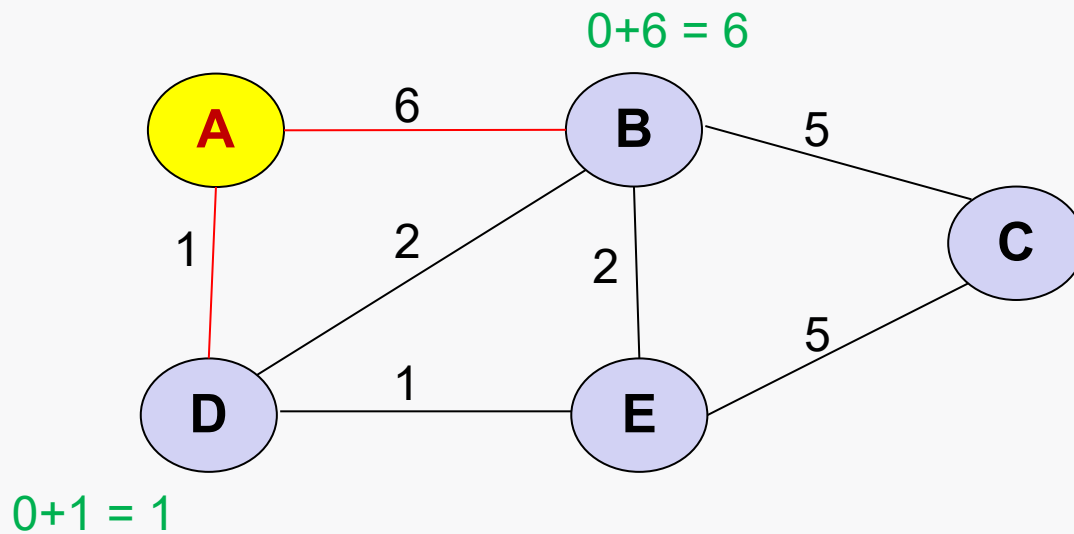
Vértice	Menor Dist. de A	Vértice Anterior
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

visitados: []

não-visitados: [A, B, C, D, E]

Dijkstra em Ação (2/7)

- PASSO 4:** Calculamos a distância de cada vizinho para o nó A



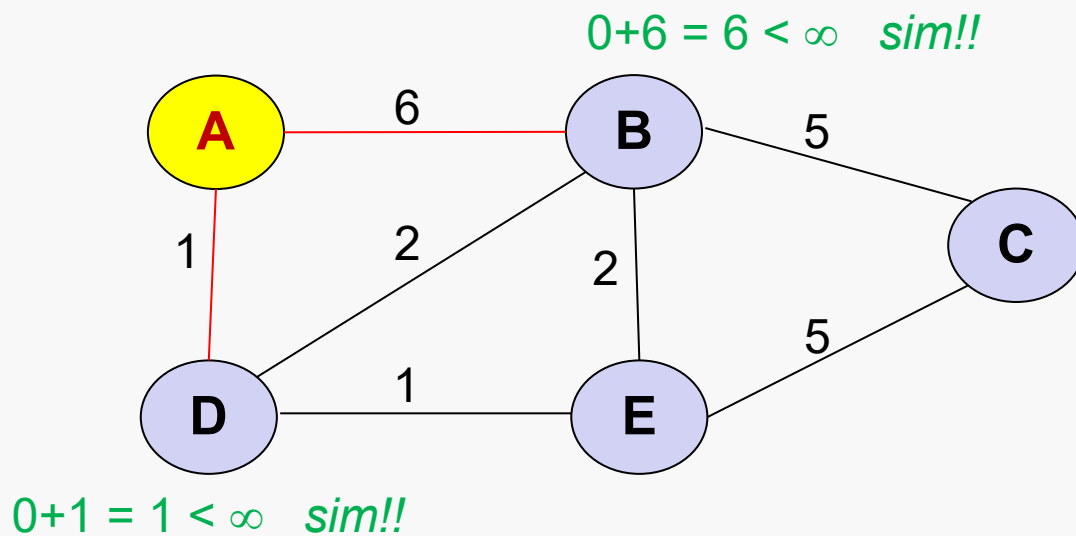
Vértice	Menor Dist. de A	Vértice Anterior
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

visitados: []

não-visitados: [A, B, C, D, E]

Dijkstra em Ação (2/7)

- **PASSO 5:** Se a distância calculada for menor do que a atualmente conhecida, a tabela precisará ser atualizada.
- Assim, descobrimos que, *até o momento*:
 - menor distância de A a B tem custo 6, via A
 - menor distância de A a D tem custo 1, via A



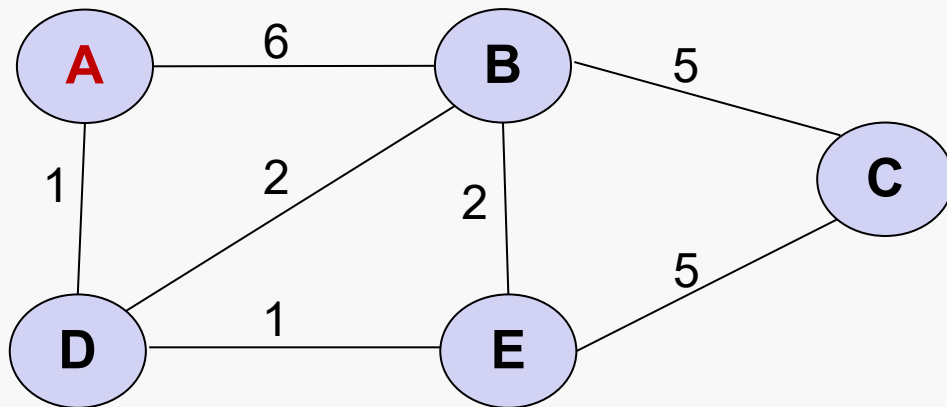
Vértice	Menor Dist. de A	Vértice Anterior
A	0	
B	∞ 6	A
C	∞	
D	∞ 1	A
E	∞	

visitados: []

não-visitados: [A, B, C, D, E]

Dijkstra em Ação (2/7)

- **PASSO 6:** Adiciona-se o nó atual A a lista de vértices visitados.
 - O nó A foi inteiramente processado e não será mais examinado
 - O algoritmo volta ao passo 2 para uma nova rodada.



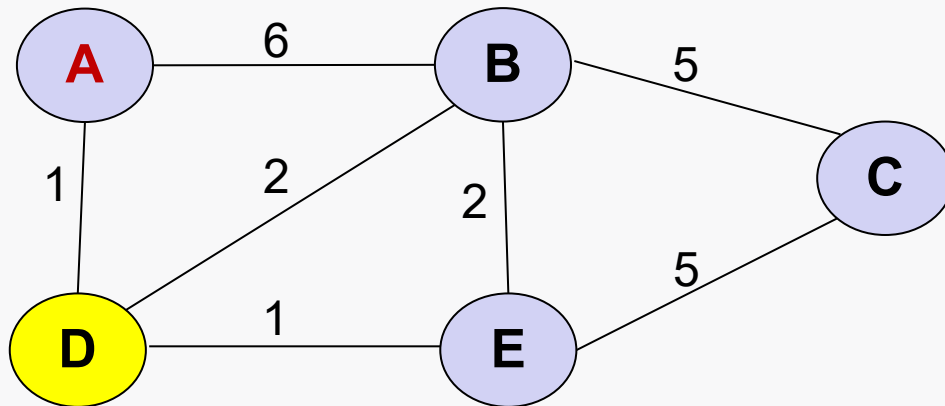
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	6	A
C	∞	
D	1	A
E	∞	

visitados: [A]

não-visitados: [B, C, D, E]

Dijkstra em Ação (3/7)

- **PASSO 2:** visitar o vértice **não visitado** que possuir a **menor distância conhecida** do vértice inicial.
 - Trata-se do vértice **D**.
 - “D” se torna o “nó atual”.



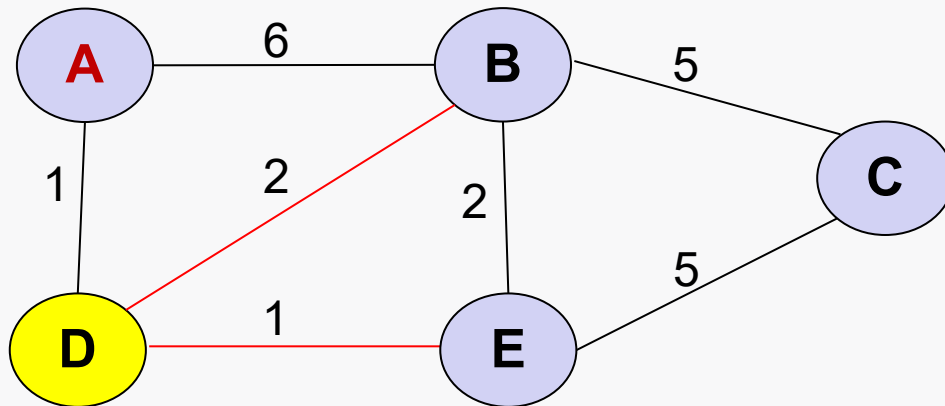
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	6	A
C	∞	
D	1	A
E	∞	

visitados: [A]

não-visitados: [B, C, D, E]

Dijkstra em Ação (3/7)

- PASSO 3:** Examina-se os vizinhos não-visitados do nó atual.
 - O nó atual é D e seus vizinhos não-visitados são B e E



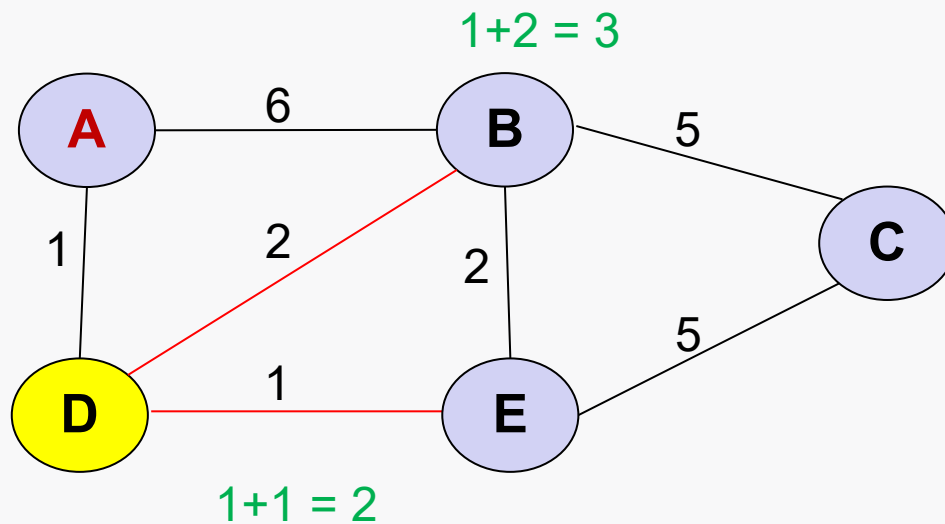
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	6	A
C	∞	
D	1	A
E	∞	

visitados: [A]

não-visitados: [B, C, D, E]

Dijkstra em Ação (3/7)

- PASSO 4:** Calculamos a distância de D para seus nós vizinhos, a partir de A (vértice anterior de D).



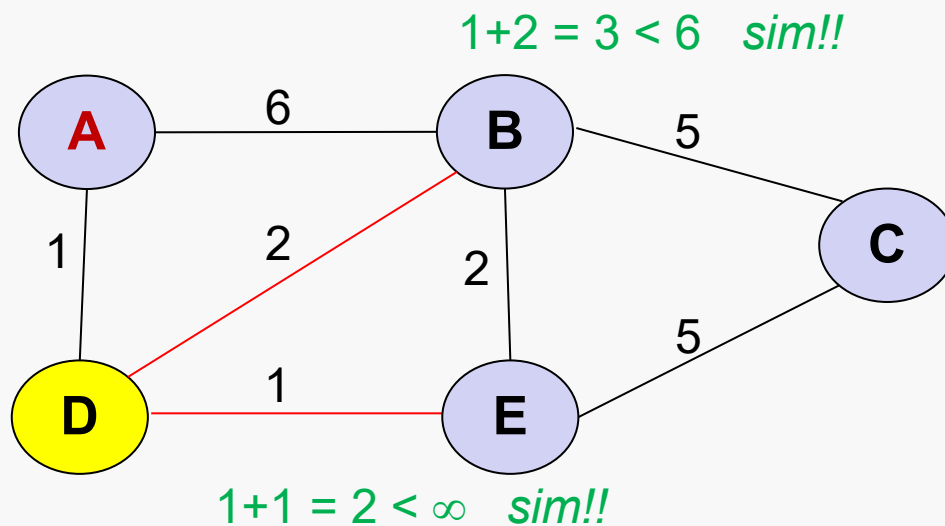
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	6	A
C	∞	
D	1	A
E	∞	

visitados: [A]

não-visitados: [B, C, D, E]

Dijkstra em Ação (3/7)

- **PASSO 5:** Se a distância calculada for menor do que a atualmente conhecida, a tabela precisará ser atualizada.
- **PASSO 6:** Adiciona-se o nó atual D a lista de vértices visitados.
 - O vértice D foi inteiramente processado e não será mais examinado
 - O algoritmo volta ao passo 2 para uma nova rodada.



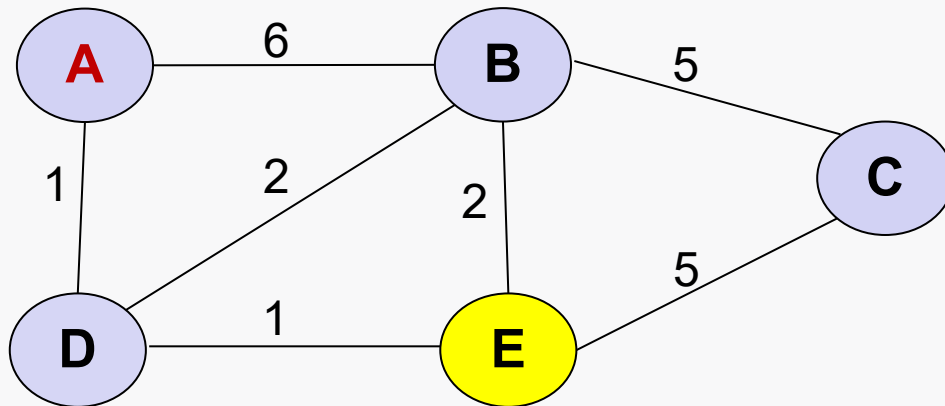
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	6 3	A D
C	∞	
D	1	A
E	∞ 2	D

visitados: [A, D]

não-visitados: [B, C, E]

Dijkstra em Ação (4/7)

- **PASSO 2:** visitar o **vértice não visitado** que possuir a **menor distância conhecida** do vértice inicial.
 - Trata-se do vértice **E**.
 - “E” se torna o “nó atual”.



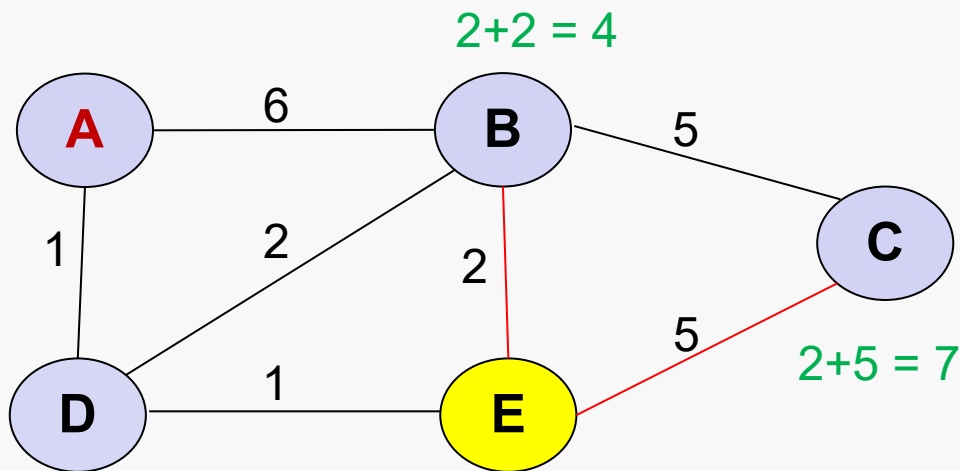
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	∞	
D	1	A
E	2	D

visitados: [A, D]

não-visitados: [B, C, E]

Dijkstra em Ação (4/7)

- **PASSO 3:** Examina-se os vizinhos não-visitados do nó atual.
- **PASSO 4:** Calculamos a distância de E para seus nós vizinhos, a partir do nó D (vértice anterior de E).



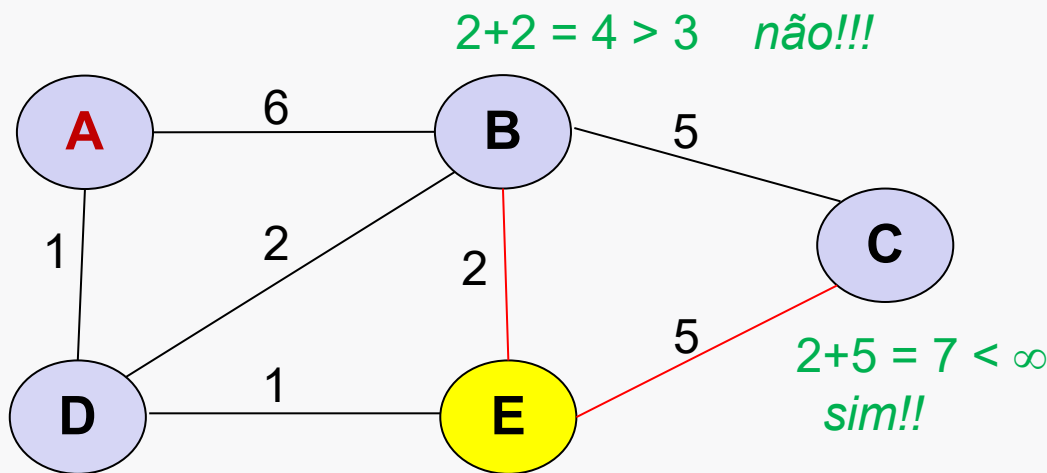
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	∞	
D	1	A
E	2	D

visitados: [A, D]

não-visitados: [B, C, E]

Dijkstra em Ação (4/7)

- **PASSO 5:** Atualiza-se a tabela, se necessário.
- **PASSO 6:** Adiciona-se o nó E a lista de vértices visitados.
 - *E voltamos ao passo 2*



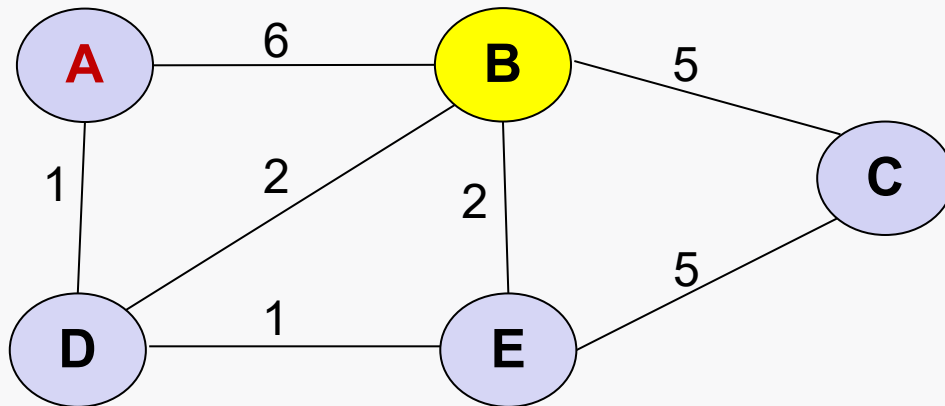
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	∞ 7	E
D	1	A
E	2	D

visitados: [A, D, E]

não-visitados: [B, C]

Dijkstra em Ação (5/7)

- **PASSO 2:** visitar o **vértice não visitado** que possuir a **menor distância conhecida** do vértice inicial.
 - Trata-se do vértice **B**.
 - “B” se torna o “nó atual”.



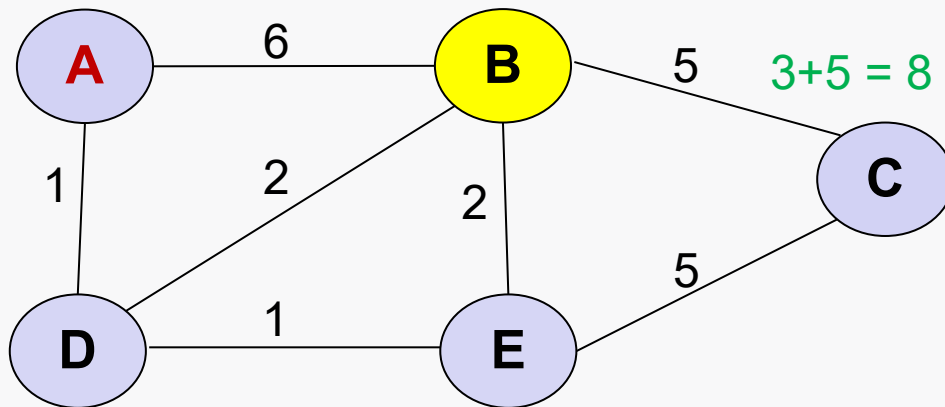
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

visitados: [A, D, E]

não-visitados: [B, C]

Dijkstra em Ação (5/7)

- **PASSO 3:** Examina-se os vizinhos não-visitados do nó atual.
- **PASSO 4:** Calculamos a distância de B para seus nós vizinhos, a partir do nó D (vértice anterior de B).



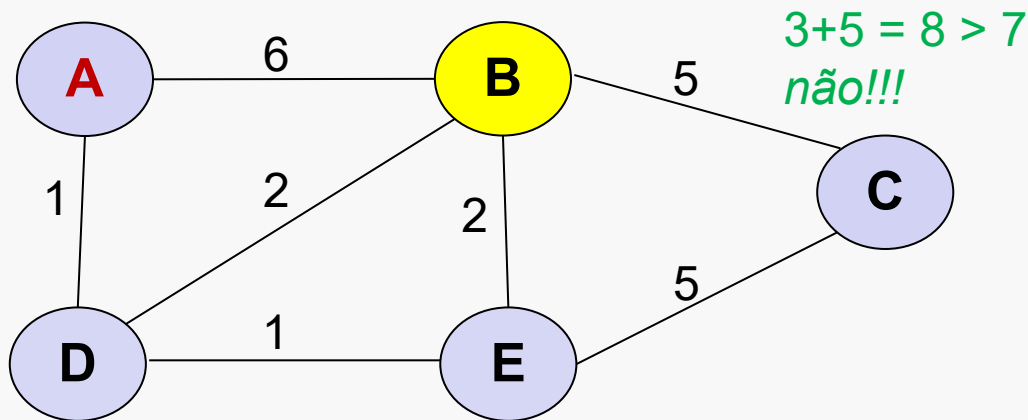
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

visitados: [A, D, E]

não-visitados: [B, C]

Dijkstra em Ação (5/7)

- **PASSO 5:** Atualiza-se a tabela, se necessário.
- **PASSO 6:** Adiciona-se o nó B a lista de vértices visitados.



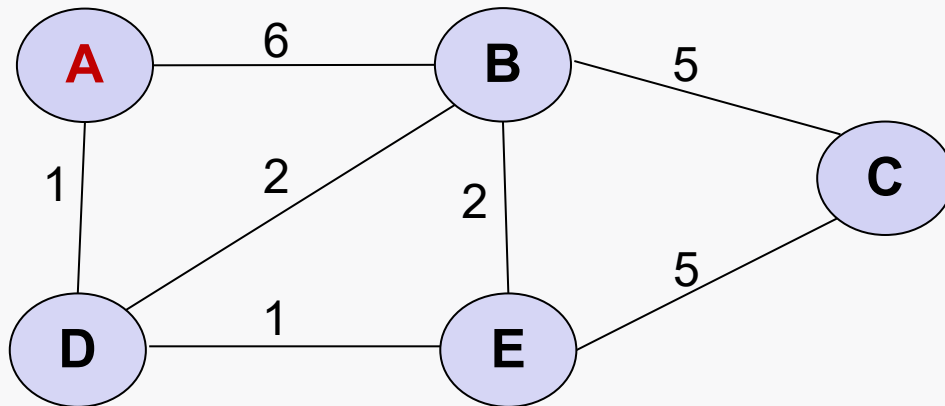
Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

visitados: [A, D, E, B]

não-visitados: [C]

Dijkstra em Ação (6/7)

- O **único** nó **não-visitado** é C.
 - Logo, C não possui nenhum vizinho não-visitado.
 - Assim o algoritmo pode terminar!!!!



Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D

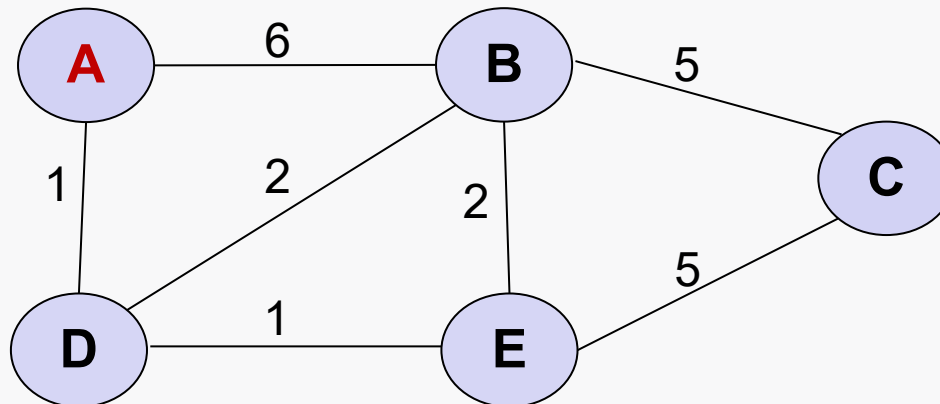
visitados: [A, D, E, B]

não-visitados: [C]

Dijkstra em Ação (7/7)

- Resultado final:

Vértice	Menor Dist. de A	Vértice Anterior
A	0	-
B	3	D
C	7	E
D	1	A
E	2	D



Pseudocódigo

Let distance of start vertex from start vertex = 0

Let distance of all other vertices from start = ∞ (infinity)

WHILE vertices remain unvisited

 Visit unvisited vertex with smallest known distance from start vertex (call this 'current vertex')

 FOR each unvisited neighbour of the current vertex

 Calculate the distance from start vertex

 If the calculated distance of this vertex is less than the known distance

 Update shortest distance to this vertex

 Update the previous vertex with the current vertex

 end if

 NEXT unvisited neighbour

 Add the current vertex to the list of visited vertices

END WHILE

Notebook com exemplo

- O link abaixo mostra um código que reproduz o exemplo da aula utilizando o pacote **networkx**
 - <https://colab.research.google.com/drive/1TDrrUwfgQH2p7OfAvkoV93VYRu5Wpw4k?usp=sharing>
- O networkx é um pacote para trabalhar com grafos em Python que possui vários algoritmos implementados, inclusive DIJKSTRA